Incident Triage RAG Project Report

Colin Vitols | IT7075 Applied AI for Cybersecurity
University of Cincinnati | October 26, 2026

Project folder: GitHub incident triage RAG project
https://github.com/vitolscolin/IT7075-Applied-AI-for-Cybersecurity/tree/rag-project/rag_project

Video: https://youtu.be/xpWqbe0acow?si=Svsk99xHfkICu-Jh

Project overview

For this project, I built an offline RAG system around five documents on cybersecurity incident triage. I wanted it to find useful evidence for common analyst questions and recognize when the documents did not have an answer. With the default settings, it found the expected evidence for all six answerable questions and refused the other two. That is a good result for this small test, but eight questions are not enough to show that it would handle incident-response questions reliably in general.

How the system works

Prepare and index. I start by breaking the documents into smaller, overlapping pieces. The split_into_chunks function is in Part 1. Part 6 loads my documents and test questions, then checks that each expected evidence phrase actually appears in its source. In Part 7, I use 500-character chunks with 80 characters of overlap. This turns the 10,209 characters of source text into 26 chunks. The overlap helps keep useful information together when it falls near the edge of a chunk.

Chunk size involves a tradeoff. If the chunks are too small, they can lose context or cut a useful phrase in half. If they are too large, several topics end up represented by one vector, which can make the search less precise. My chunker counts characters rather than following sentence boundaries, so some chunks begin or end in the middle of a sentence. This shows up in the saved responses.

Embed. Part 1 loads all-MiniLM-L6-v2 on the CPU. An embedding turns text into a list of numbers that represents patterns in its meaning. Related pieces of text tend to have similar vectors, which lets the system search by meaning instead of requiring the question to use the exact words in the document. A close match is still only a clue, though. It does not guarantee that the text contains the answer.

Store. In Part 2, build_collection puts each chunk into Chroma along with its normalized embedding, an ID, and the source filename. The collection stays in memory and is rebuilt when the notebook runs. I use cosine distance to compare vectors. A smaller distance means a closer match.

From a question to a response

Retrieve. The retrieve function in Part 3 embeds the question with the same model used for the documents. Chroma then returns the nearest chunks, their distances, and their source information. My default settings request up to three chunks and keep only those with a distance of 0.50 or less. If none are close enough, the system has no context to use for an answer.

Augment. In Part 4, build_context and ask arrange the retrieved chunks into numbered blocks with source filenames. Those blocks go into a prompt that tells the assistant to answer from the supplied text, cite its sources, and say when it does not have enough information.

Generate or refuse. My notebook uses an offline fallback in grounded_answer, also in Part 4. It shows up to 360 characters from the first retrieved chunk and names the source. If no chunks pass the cutoff, it returns a refusal. The notebook builds the prompt, but it does not send it to a language model. This means I am testing retrieval and an evidence-based response, not how well an LLM writes an answer or follows instructions.

Part 5 scores the practice pipeline, which produces the required 22 chunks, 6/6 evidence matches, and 2/2 refusals. Part 7 prints custom transcripts for Q1, Q3, Q6, Q7, and Q8. For example, Q1 retrieves the instruction to preserve failed and successful sign-ins as one timeline. Q8 returns no retained chunks and responds, “I don't know; the knowledge base does not contain relevant evidence.”

Documents and question design

I organized the knowledge base into five files: account_triage.md, endpoint_triage.md, email_triage.md, network_triage.md, and evidence_and_escalation.md. They cover account activity, process evidence, suspicious email, network correlation, and evidence handling. SOURCES.md records that these are original project procedures using made-up examples, not records from real incidents.

I stored the eight test questions in goldset.json. For each answerable question, I included a short phrase copied exactly from the relevant document. Part 6 checks those phrases before the system is scored. I leave SOURCES.md out of the index because it describes where the documents came from rather than providing triage guidance.

I used two different kinds of unanswerable questions. Q7 asks about watering tomato seedlings, which is clearly outside the topic. Q8 asks for the exact SIEM retention period required by the triage program. I chose that question because the documents mention retention rules but never give a specific period. The system needs to recognize that related wording is not enough to answer it. That makes Q8 a more useful challenge than a question that is obviously unrelated.

The eight evaluation questions

Q1. How should failed sign-ins followed by a successful sign-in be preserved?

Expected evidence: preserve the failed and successful sign-in events as one timeline
Source: account_triage.md

Q2. When a workstation downloads and launches an executable, what process evidence should be recorded before host isolation?

Expected evidence: record the parent-child process chain before isolating the host
Source: endpoint_triage.md

Q3. What should an analyst preserve from a suspected phishing email?

Expected evidence: preserve the original message with full headers and attachment hashes
Source: email_triage.md

Q4. Which data sources and time standard should be correlated for command-and-control traffic?

Expected evidence: correlate DNS, proxy, firewall, and endpoint timestamps in UTC
Source: network_triage.md

Q5. How can an analyst verify that an exported artifact did not change?

Expected evidence: Use SHA-256 to verify that an exported artifact has not changed
Source: evidence_and_escalation.md

Q6. What must remain labeled unresolved in the incident case record?

Expected evidence: An unresolved cause must remain labeled unresolved in the case record
Source: evidence_and_escalation.md

Q7. What is the recommended watering schedule for tomato seedlings?

Expected outcome: Refuse because the documents do not answer this question.

Q8. What exact SIEM retention period does this incident-triage program require?

Expected outcome: Refuse because the documents do not answer this question.

What the scores measure

The found count tells me whether any retrieved chunk contains the expected evidence phrase, ignoring capitalization. The refused count tells me how many unanswerable questions returned no chunks. These checks do not tell me whether the displayed response is complete. The balanced_score column in results.csv adds the answer rate and refusal rate, giving each equal weight and producing a score from 0 to 2.

Experiment results

In Part 8, I compare settings by changing one at a time and keeping overlap at 80 characters throughout. Part 9 creates the chart, while results.csv stores the scores and the IDs of missed questions. The table and chart below come from the full notebook run on October 26, 2026.

Configuration	Size	k	Cutoff	Chunks	Found	Refused
default	500	3	0.5	26	6/6	2/2
chunk_350	350	3	0.5	41	5/6	2/2
chunk_700	700	3	0.5	20	4/6	2/2
k_1	500	1	0.5	26	5/6	2/2
k_5	500	5	0.5	26	6/6	2/2
threshold_0.35	500	3	0.35	26	2/6	2/2
threshold_0.65	500	3	0.65	26	6/6	1/2
threshold_0.80	500	3	0.8	26	6/6	1/2

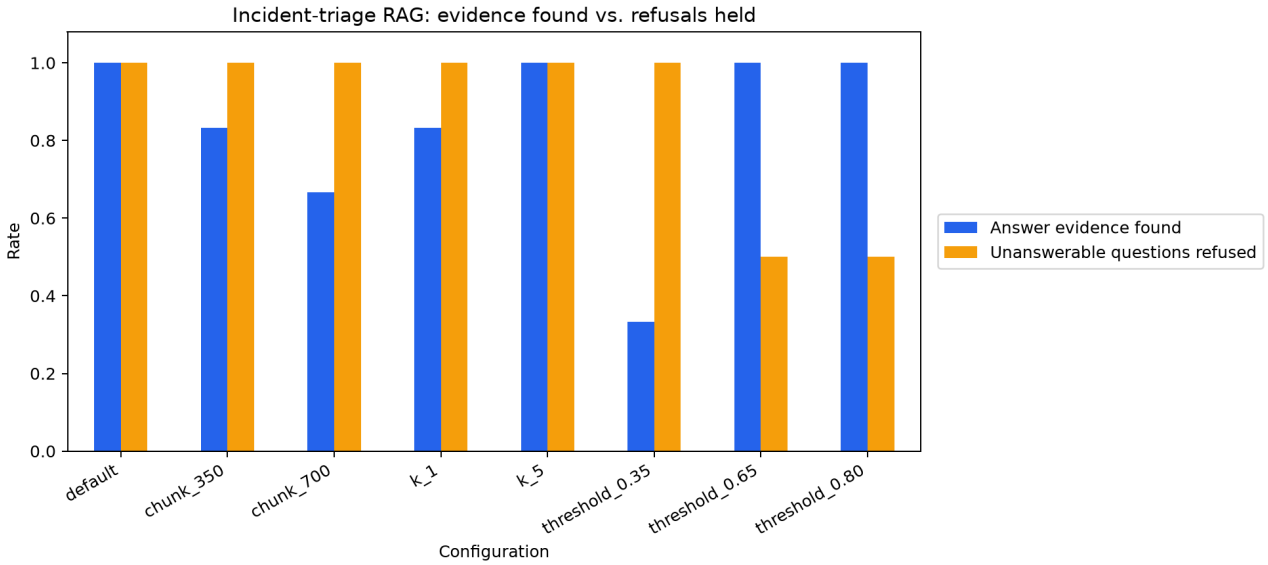


Figure 1. Evidence retrieval and refusal rates for each configuration.

The distance cutoff made the biggest difference. Lowering it to 0.35 kept both refusals but found evidence for only 2/6 answerable questions. Raising it to 0.65 or 0.80 found all six, but the system stopped refusing Q8. It retrieved related triage text even though that text did not give the requested retention period. Getting more matches was not helpful when one of those matches should have been a refusal.

Based on these results, I would keep 500-character chunks, 80-character overlap, k=3, and a cutoff of 0.50. The 350-character chunks found 5/6 answers, and the 700-character chunks found 4/6. Using k=1 missed Q3, while k=5 gave the same scores as the default. I would stay with k=3 because retrieving more chunks did not improve the results here. I would test these settings again if I added more documents or different questions.

A retrieval mistake

In Part 10, I looked more closely at Q3: “What should an analyst preserve from a suspected phishing email?” With k=1, the system chose the first chunk from email_triage.md, with a cosine distance of about 0.404. It starts with “Suspicious Email Triage” and “Preserve the message.” It also says to preserve the original message or an approved export so headers, attachment hashes, URLs, and routing details remain available.

The selected chunk is relevant, but it does not contain the exact phrase used for scoring: “preserve the original message with full headers and attachment hashes.” That phrase is in the second-ranked chunk, at a distance of about 0.407. With k=1, the system never gets that second chunk, so Q3 counts as a miss. With the default k=3, both chunks pass the 0.50 cutoff and the evidence check succeeds.

What stood out to me is how close the two distances are. The model puts a useful introductory passage just ahead of the passage with the exact evidence. The first chunk also gives much of the right information in different words, so I would not call the response completely wrong. It fails the exact-phrase check. There is another limitation here: the offline response only displays the first chunk. Finding the gold phrase in a later chunk improves the retrieval score, but does not mean that phrase appears in the response the user sees.

Q1 shows a different problem. Its correct account-triage chunk has a distance of about 0.410, so the stricter 0.35 cutoff removes it. The system then refuses a question that the documents can answer. Part 10 prints that chunk and the next two matches, making it clear that the evidence was available but filtered out.

Limits and next steps

The main limitation is the size of the test. With only six answerable questions, one missed question changes the answer rate by about 16.7 percentage points. With only two refusal questions, one mistake changes that rate by 50 points. I also used the same questions to compare settings, and the questions were designed around these documents. A perfect score here does not tell me how well the system will handle new questions.

I have not tested reworded or ambiguous questions, conflicting procedures, larger document collections, prompt injection, or real incidents. The exact-phrase check can miss a valid answer that uses different wording, and chunk boundaries can split a phrase. It can also count a question as successful when the evidence is retrieved but left out of the displayed response. The distance cutoff helps reject weak matches, but Q8 shows that related text can still pass without answering the question.

Next, I would add questions that were not used to choose the settings, including reworded questions and more questions that sound relevant but cannot be answered from the files. I would also try splitting text at sentence boundaries and score the completeness of the displayed responses separately. If I connect an LLM, I would need to check its answers for factual support, citations, and appropriate refusals. For now, this project shows how a small offline retrieval pipeline works and where it can make mistakes.

Project evidence: rag_notebook.ipynb Parts 1-10; goldset.json; results.csv; figure.png; my_kb/SOURCES.md and the five procedure files.